SWE 402 - Game Development
Assignment 2: 2D Roguelike or 3D Haunted House

Overview

Choose ONE option for this assignment. Complete either the 2D Roguelike Game or the 3D Stealth Game: Haunted House. Follow the sections for your chosen option; you do not need to complete both games.

Due Date & Submission

Monday, October 19, 2026 at 2:00 PM (Riyadh time, UTC+3).

Submit a GitHub repository link on Blackboard containing your complete Unity project and a proper README.md. Identify your chosen option, explain how to open and run the game, and list any bonus features implemented. Include the build or published-game information required by your chosen option.

Option 1: 2D Roguelike Game

Build a turn-based roguelike with procedural level generation, enemies, and resource management. Complete the course and the additional requirements below.

Course Link: Create a 2D Roguelike Game

Option 2: 3D Stealth Game: Haunted House

Build a complete 3D stealth game with player movement, animation, static and dynamic enemies, game-ending conditions, and audio. Follow the Haunted House sections later in this document.

Course Link: 3D Stealth Game: Haunted House

Requirements and bonus rules are specific to each option. The Roguelike enhancements and three-platform builds apply only to Option 1. The deadline and GitHub submission apply to both.

Option 1: 2D Roguelike Game

Section 1: Project Setup & Game Board

Complete the tutorial(s): "Set up your project" + "Add a game board"

Objectives:

- Create a new Unity project using Editor 6.x LTS and the Universal 2D template
- Import the provided Roguelike2D tutorial assets package
- Set the Main Camera to Orthographic, size 5, solid black background
- Create a Tile Palette with ground tiles from the provided Sprite Sheets
- Write a BoardManager script that procedurally generates an 8×8 tilemap using nested for-loops
- Use Tile[] arrays and Random.Range() to randomly assign ground and wall tiles
- Add border detection logic so edge cells use wall tiles
- Create a CellData nested class with a Passable boolean; store board state in a CellData[,] 2D array
- Set up Unity Version Control and check in your initial changes

Section 2: Player Character & Movement

Complete the tutorial(s): "Add a player character"

Objectives:

- Create a PlayerCharacter GameObject from a provided sprite; set Order in Layer for correct rendering
- Write a PlayerController script that reads arrow key input via Input.GetKeyDown()
- Implement grid-based (cell-to-cell) movement with passability checks using GetCellData()
- Create a Spawn() method to initialize the player at a given cell coordinate
- Refactor BoardManager: rename Start() to Init() and make it public for external initialization

Section 3: Turn System & Game Architecture

Complete the tutorial(s): "Add a turn system"

Objectives:

- Create a TurnManager as a pure C# class (non-MonoBehaviour) with a constructor and Tick() method
- Create a GameManager MonoBehaviour that initializes TurnManager, BoardManager, and PlayerController
- Implement the Singleton pattern on GameManager using static Instance with public get; private set;
- Use Awake() for singleton enforcement and Start() for game initialization
- Call TurnManager.Tick() from PlayerController each time the player moves

Section 4: Food Resource & Inheritance

Complete the tutorial(s): "Add a food resource"

Objectives:

- Create a CellObject base class with a virtual PlayerEntered() method
- Create a FoodObject subclass that overrides PlayerEntered() to grant food and destroy itself
- Store a ContainedObject reference in CellData so cells know what objects they hold
- Add food tracking to GameManager; decrease food each turn, increase on collection
- Use UI Toolkit (UI Builder window) to create a FoodLabel that displays the current food count
- Create multiple food prefabs with different AmountGranted values; randomize spawns at level start

Section 5: Obstacles, Win/Lose Conditions & Levels

Complete the tutorial(s): "Add obstacles" + "Add win and lose conditions"

Objectives:

- Create a WallObject subclass of CellObject that changes the tile sprite and blocks player movement
- Implement PlayerWantsToEnter() pattern — walls take damage on bump and are destroyed at 0 HP
- Create an ExitCellObject subclass that triggers level completion when the player enters its cell
- Implement level progression: clear the board and regenerate a new level on exit
- Add a Game Over state when food reaches 0 — disable input and display a Game Over panel via UI Toolkit
- Implement game restart functionality on key press to return to level 1

Section 6: Animation & Enemies

Complete the tutorial(s): "Animation" + "Add an enemy"

Objectives:

- Add an Animator component to the player with an AnimatorController containing Idle, Walk, and Attack states
- Create sprite-based keyframe animations using the Animation window; set sample rate appropriately
- Configure state transitions with proper Has Exit Time, Transition Duration = 0, and Bool/Trigger conditions
- Control animations from code using SetBool() and SetTrigger()
- Create an Enemy class inheriting from CellObject that subscribes to TurnManager.OnTick using += and unsubscribes in OnDestroy() using -=
- Implement enemy AI: move toward the player each turn; attack (reduce food) if adjacent
- Enemy blocks player entry, takes damage on bump, and is destroyed at 0 HP (same pattern as WallObject)

Section 7: Additional Requirements (Beyond the Course)

Objectives:

After completing all course tutorials, implement all of the following enhancements to your project. These integrate concepts from previous missions (M1–M6) that the Roguelike course does not cover.

Audio (M3 concepts)

- Add background music via an AudioSource on the Main Camera (loop enabled, reduced volume)
- Add sound effects using PlayOneShot() for: player movement, wall attack, food pickup, enemy attack, enemy death, and game over

Visual Effects (M3 concepts)

- Add ParticleSystem effects for at least 3 events: wall destruction, food collection, and enemy death
- Control particles from code using .Play() and .Stop(); instantiate or pool particle prefabs as needed

Smooth Cell Movement (M3–M5 concepts)

- Replace instant grid snapping with smooth movement using a Coroutine that lerps the player between cells
- Block input during the movement coroutine to prevent skipping cells

Object Pooling (M6 concepts)

- Implement object pooling for CellObjects (food, walls, enemies) — pre-instantiate a pool at game start
- On level transition, recycle objects via SetActive(false/true) instead of Destroy/Instantiate

Code Standards (M6 concepts)

- Use [SerializeField] on all private variables exposed to the Inspector — no unnecessary public fields
- Use [Range()] on numeric tuning variables (e.g., speeds, spawn counts) and [Tooltip()] where helpful

Section 8: Build & Submit

Objectives:

Build your completed project for all three platforms and submit.

- Install Windows, macOS, and WebGL build modules via Unity Hub
- Add your scene(s) to File → Build Profiles → Scene List
- Build for Windows (standalone .exe), macOS (.app), and WebGL (index.html + supporting files)
- Test each build to verify it runs correctly outside the Editor
- (Optional) Upload your WebGL build to Unity Play or itch.io for browser testing

Submission: Submit the GitHub repository link as described above. Include the Builds/ folder with Windows, macOS, and WebGL builds.

Bonus Features (Optional — Up to 10 Extra Points)

Implement any of the following for extra credit. Each feature is worth up to 5 points.

- Dynamic Board Scaling: Increase board size and/or number of enemies/walls as the player progresses to higher levels
- Multiple Enemy Types: Create 2+ enemy subclasses with different AI behaviors (e.g., one chases, one patrols, one ambushes)
- Items & Stats System: Add collectible items (weapons, armor, potions) that modify player stats using ScriptableObjects
- Save & Load System: Implement data persistence across sessions using JSON serialization (PlayerPrefs or file I/O)
- Main Menu & Pause Menu: Add a title screen with Play/Quit buttons and an in-game pause menu with Resume/Restart/Quit
- Custom Sprites & Theme: Replace all provided assets with your own sprites or a cohesive custom theme

Resources

- Unity Learn Course: Create a 2D Roguelike Game
- Unity 6 Documentation
- Unity Manual: Tilemaps
- Unity Manual: UI Toolkit
- Unity Learn: Introduction to Object Pooling
- Free Assets: Unity Asset Store, OpenGameArt.org, Kenney.nl

Option 2: 3D Stealth Game: Haunted House

Overview

Complete the Haunted House course to create a playable 3D stealth game. Use the linked tutorials for the detailed implementation steps. The objectives below summarize the work to complete and verify.

Course Link: 3D Stealth Game: Haunted House

Section 1: Project Setup & Game World

Complete the tutorial: "Scene setup"

Objectives:

- Create the Unity project and import the assets used by the course
- Set up and save the haunted-house scene
- Arrange the environment and identify the playable area
- Organize the player, environment, and enemy objects in the Hierarchy

Section 2: Player Controls

Complete the tutorial: "Set up Player movement"

Objectives:

- Add and configure the player character
- Implement the movement controls demonstrated in the tutorial
- Make the player turn to face the movement direction
- Test movement around the environment and collisions with obstacles

Section 3: Player Animation

Complete the tutorial: "Animate your Player character"

Objectives:

- Set up the player animation components as taught in the course
- Connect the animation behavior to player movement
- Verify the character animates while moving and returns to idle when stopped

Section 4: Camera Setup

Complete the tutorial: "Set up your camera"

Objectives:

- Position and configure the gameplay camera
- Implement the camera behavior demonstrated in the tutorial
- Test that the player and nearby obstacles remain visible during gameplay

Section 5: Static Enemies

Complete the tutorial: "Create static enemies"

Objectives:

- Add the static enemy objects to the scene
- Configure their detection areas and player detection behavior
- Verify that the player can avoid detection by navigating safely
- Test that entering a static enemy detection area triggers the expected response

Section 6: Dynamic Enemies

Complete the tutorial: "Create dynamic enemies"

Objectives:

- Add the moving enemies and configure their movement as taught in the tutorial
- Set up their patrol or movement routes
- Connect their detection behavior to the player
- Test that enemies move through the environment and detect the player correctly

Section 7: Win & Lose Conditions

Complete the tutorial: "Ending the game"

Objectives:

- Set up successful escape and being-caught conditions
- Connect static and dynamic enemy detection to the losing condition
- Display the game-ending feedback taught in the tutorial
- Test both a successful escape and an unsuccessful attempt

Section 8: Sound Effects & Background Audio

Complete the tutorial: "Add audio to your game"

Objectives:

- Add and configure the audio components used by the course
- Connect the demonstrated sounds to the appropriate gameplay events
- Adjust audio levels so the sounds are clear during play
- Test audio during movement, enemy encounters, and game-ending sequences

Section 9: Build & Publish

Complete the exercise: "Publish and share your game"

Objectives:

- Configure the scene and build settings as instructed in the course
- Build and publish the completed game using the course workflow
- Test the built or published game outside the Editor
- Include the build or published-game link and run instructions in your README.md

Bonus Features (Optional)

Complete the course tutorial "Bonus features" if you want to extend or personalize your Haunted House game. Document any extensions in your README.md. The Option 1 bonus-point scheme applies only to the Roguelike option.

Resources

Unity Learn Course: 3D Stealth Game: Haunted House

Unity Documentation: https://docs.unity3d.com/

Unity Play: https://play.unity.com/